Experiment No: 5			
Course Name:	Web Programming Laboratory (216U01L405)	**Semester:**	**IV**
Date of Performance:	**30/03/26**	**DIV/ Batch No:**	**B2**
Student Name:	**ASHWERA HASAN**	**Roll No:**	**16010124107**

Title: Design and Develop a web application that integrates PHP and HTML to display dynamic content based on user inputs and server-side data.

Aim of the Experiment:

The aim of this experiment is to design and develop a web application that utilizes PHP and HTML to create interactive, dynamic web pages. This application will allow users to input data through a form, which is then processed on the server side to display customized content based on the user's inputs.

Objectives for the Experiment:

- To Implement User Input Handling: Develop an HTML form that allows users to input data, ensuring a user-friendly interface for data collection.
- To Integrate PHP for Server-Side Processing: Utilize PHP to process the data received from the HTML form, enabling dynamic content generation based on user inputs.
- To Dynamically Display Results: Create a method to display the processed information on the web page, showcasing how server-side scripts can influence the content presented to the user.

COs to be achieved:

1. Design dynamic web pages using various HTML tags.
2. Use CSS to prepare the layout of web pages.
3. Apply JavaScript for validation in client-side programming.
4. Integrate server-side pages using PHP.

Books/ Journals/ Websites references:

1. Welling, L., & Thomson, L. (2017). PHP and MySQL Web Development (5th ed.). Pearson.
2.
3. Duckett, J. (2014). PHP & MySQL: Server-side Web Development. Wiley.

Methodology:

Setting Up the Server Environment

- Install a local server environment (XAMPP/MAMP) to host the application.
- Start the Apache server and ensure it is running.

Creating HTML Form

- Develop an HTML form to receive user input.

Processing User Input with PHP

- Create a PHP file (process.php) to handle the form submission.

Testing the Application

- Open the HTML file in a web browser.
- Fill in the form and submit to ensure that the output displays correctly and dynamically based on user input

Problem statement:

Design and implement an application to demonstrate HTML form integration with PHP for data collection and processing.

Utilize the registration page designed in Experiment No. 1 and create PHP scripts to handle form submission and data processing as follows:

1. Create separate PHP scripts for handling form submissions using different methods:

   - **post_registration.php**: Processes the registration form data using the `$_POST` method.
   - **get_registration.php**: Handles the form data using the `$_GET` method.
   - **request_registration.php**: Retrieves the submitted data using the `$_REQUEST` method.

2. Each script should validate the input fields (e.g., check for valid email format and ensure mandatory fields are filled) and display the submitted registration details in a structured format.

Also,

1. **Define Problem statement for your Mini Project.**
2. **Start working on Figma design of Mini Project.**
3. **Design Frontend pages.**
4. **Apply validation over**

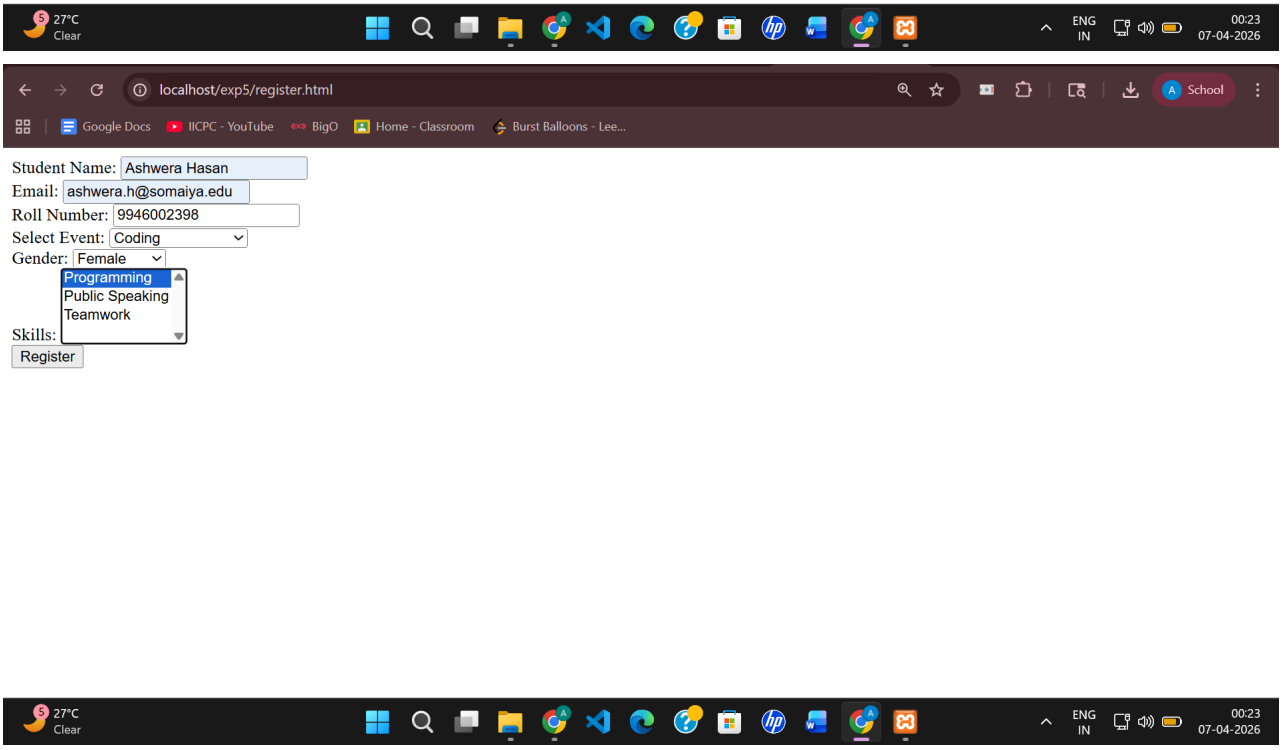
Code:

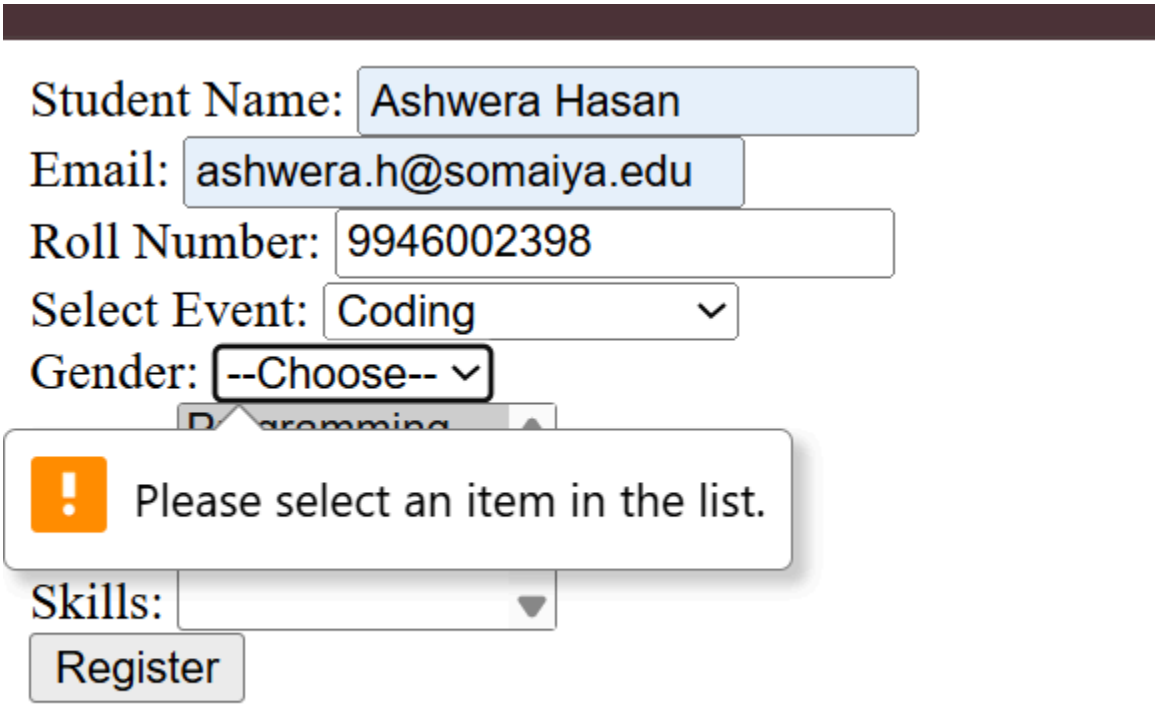
Link: https://github.com/ashwera/webProgrammingLab/tree/main/experiment%205

Output / Screenshot:

Registration Details (POST)

Name: Ashwera Hasan
Email: ashwera.h@somaiya.edu
Roll: 9946002398
Event: coding
Gender: female
Skills: programming





Post Lab Subjective/Objective type Questions:

1. Which method ($_POST, $_GET, $_REQUEST) is the most secure, and why?
   $_POST is the most secure because data is sent in the request body, not visible in the URL. $_GET exposes data in URLs, making it less secure. $_REQUEST is least secure as it combines multiple sources. However, validation and HTTPS are still required.

2. From a developer's perspective, which method ($_POST, $_GET, $_REQUEST) is easier to use, and why?
   From a developer's perspective, $_REQUEST is easiest because it retrieves data from GET, POST, and COOKIE in one place, reducing effort. However, it is less secure and less predictable, so $_POST or $_GET are preferred for clarity and control.

3. How does the $_GET method handle data transmission, and what are its limitations?
   $_GET sends data through the URL as query parameters (key=value). It is simple and fast, useful for retrieving data. Limitations include URL length restriction, visible data (less secure), data stored in browser history, and not suitable for sensitive information.

4. Write a PHP code snippet that sanitizes user inputs using htmlspecialchars() or similar functions to prevent XSS vulnerabilities. How would you ensure that inputs are safe to display?

```php
<?php
function sanitize($data) {
    $data = trim($data);
    $data = stripslashes($data);
    $data = htmlspecialchars($data, ENT_QUOTES, 'UTF-8');
    return $data;
}

$name = sanitize($_POST['name'] ?? '');
$email = sanitize($_POST['email'] ?? '');

echo "Name: " . $name . "<br>";
echo "Email: " . $email;
?>
```

Ensure safety by escaping output using htmlspecialchars(), validating input format, and always specifying UTF-8 to prevent XSS.

5. Create a function in PHP that validates the user inputs to ensure they meet certain criteria (e.g., username must contain only letters and numbers, and the user input must not exceed 200 characters).

```php
<?php
function validate_input($username) {
    if (strlen($username) > 200) {
        return "Input too long";
    }
    if (!preg_match("/^[a-zA-Z0-9]*$/", $username)) {
        return "Only letters and numbers allowed";
    }
    return "Valid input";
}
?>
```

6. Write a PHP function that generates a personalized greeting message based on the user's input. For example, if the input includes their favorite hobby, include that in the greeting.

```php
<?php
```

```php
function greeting($name, $hobby) {
    return "Hello $name! Nice to know you like $hobby.";
}
?>
```

7. Develop a JavaScript function that checks user inputs before submission. The function should confirm that all fields are filled and provide user feedback if any field is empty.

```javascript
function validateForm() {

    let inputs = document.querySelectorAll("input");

    for (let input of inputs) {

        if (input.value.trim() === "") {

            alert("All fields must be filled");

            return false;

        }

    }

    return true;

}
```

Conclusion and Discussion:

Proper validation, sanitization, and checks ensure secure, reliable input handling, prevent errors, and improve user experience in web applications.

- Form is simple and user friendly but needs better validation and clear error messages for improved usability. — Riya
- $_POST implementation is secure and reliable, but input sanitization and validation should be strengthened. — Mohammad
- $_GET and $_REQUEST work but expose data and reduce predictability, so should be used carefully. — Naitik